CoderUJB

Six intervenants · Textes oraux en français

六人讲稿与 PPT 逐页对照

分工与时间

人员与任务	PPT 页码	词数	预计时长
1 你：研究背景	1-4	407	3:54
2 贡献与方法	5-8	337	3:34
3 结果：模型表现	9-12	344	3:38
4 结果：训练影响	13-16	349	3:41
5 研究局限	17-20	346	3:40
6 评析与总结	21-24	348	3:41

时间说明

第一部分保留原讲稿与四页 PPT，按约 110 词/分钟计算；其余五部分按约 100 词/分钟计算。每人另计 12 秒停顿和换页，总时长约 22 分钟。计时为估算，不是实际录音测量。

第一部分若低于约 107 词/分钟，可能超过 4 分钟。排练超时时，可不念第 2 页从“On peut comparer cela…”到“le second.”的比喻段，约省 15 秒，不影响 PPT 或核心观点。

使用方法

每页对应一张 PPT：上方看幻灯片，下方读法语讲稿。绿色关键词和来源不需要念。每个人的最后一页已包含交接句。PPT 备注区也有完整讲稿，方便演讲者视图使用。

所有实验数字来自原论文，没有重新运行模型实验。第五、六部分明确区分作者说明与小组分析。六人分工按你们的方案制作。

INTERVENANT 1 · CONTEXTE DE LA RECHERCHE

01. Comment évaluer une IA qui code ?



TEXTE À DIRE

Bonjour à tous. Aujourd'hui, nous allons présenter l'article CoderUJB, publié en 2024. Je vais d'abord expliquer le contexte de cette étude.

Les grands modèles de langage, qu'on appelle aussi les LLM, peuvent aider à programmer. Ils peuvent, par exemple, proposer du code ou aider à corriger une erreur.

Mais comment savoir si leurs réponses sont bonnes ? [Pause : 2 secondes]

Un code peut sembler correct quand on le lit. Pourtant, il peut échouer quand on l'exécute. Il faut donc le tester.

Pour comparer les modèles, les chercheurs utilisent des benchmarks. Un benchmark, c'est un ensemble de tâches avec des règles communes pour évaluer les réponses.

Repères : IA / code / tests / benchmark

Source : Introduction, tableau 1 et §2, pp. 124-126.

02. Un exercice isolé a ses limites



TEXTE À DIRE

Un exemple connu est HumanEval. Il contient cent soixante-quatre exercices en Python. Le modèle doit compléter une fonction à partir d'une description. Ensuite, des tests vérifient le résultat.

C'est utile, car on exécute vraiment le code. [Pause : 2 secondes]

Mais les exercices restent assez isolés. Dans un vrai projet, une fonction peut utiliser d'autres fichiers ou des bibliothèques. Le modèle doit donc tenir compte du code qui existe déjà.

On peut comparer cela à un exercice de cours et à un travail dans un logiciel complet. Réussir le premier ne garantit pas qu'on réussira le second. C'est une première difficulté pour l'évaluation.

Repères : HumanEval / fonction / projet / dépendances

Source : Introduction, tableau 1 et §2, pp. 124-126.

03. Développer, c’est faire plusieurs tâches



TEXTE À DIRE

Il y a aussi une deuxième difficulté. Le travail d’un développeur comprend plusieurs types de tâches. Il faut écrire du code, mais aussi créer des tests, trouver des erreurs et les corriger.

Un modèle peut être bon pour une tâche et moins bon pour une autre. Une seule tâche donne donc une vision limitée de ses capacités. [Pause : 2 secondes]

Certains benchmarks proposent plusieurs tâches. Mais, selon les auteurs, ils ont d’autres limites. Par exemple, CodeXGLUE utilise des mesures de ressemblance entre les textes. XCodeEval utilise des problèmes de concours.

Ces approches ne réunissent donc pas toutes les conditions recherchées pour évaluer le travail dans un vrai projet.

Repères : écrire / tester / détecter / corriger

Source : Introduction, tableau 1 et §2, pp. 124-126.

04. Le besoin : une évaluation plus réaliste



TEXTE À DIRE

Les auteurs cherchent donc à réunir trois éléments dans une même évaluation.

D’abord, exécuter les solutions pour vérifier leur fonctionnement. Ensuite, utiliser de vrais projets, avec leur code et leur environnement. Enfin, proposer plusieurs types de tâches. [Pause : 2 secondes]

C’est dans ce contexte qu’ils présentent CoderUJB. Ils choisissent Java, un langage très utilisé dans le développement de logiciels.

La question de départ est donc simple : comment mieux évaluer les capacités des modèles dans des situations proches du travail réel ?

Voilà la motivation de cette étude. Je laisse maintenant la parole à la personne suivante, qui va présenter les contributions de l’article.

Repères : exécution / projets / tâches / Java

Source : Introduction, tableau 1 et §2, pp. 124-126.

05. Un benchmark commun pour cinq tâches



TEXTE À DIRE

Merci. Je vais présenter ce que les auteurs proposent et comment ils évaluent les modèles.

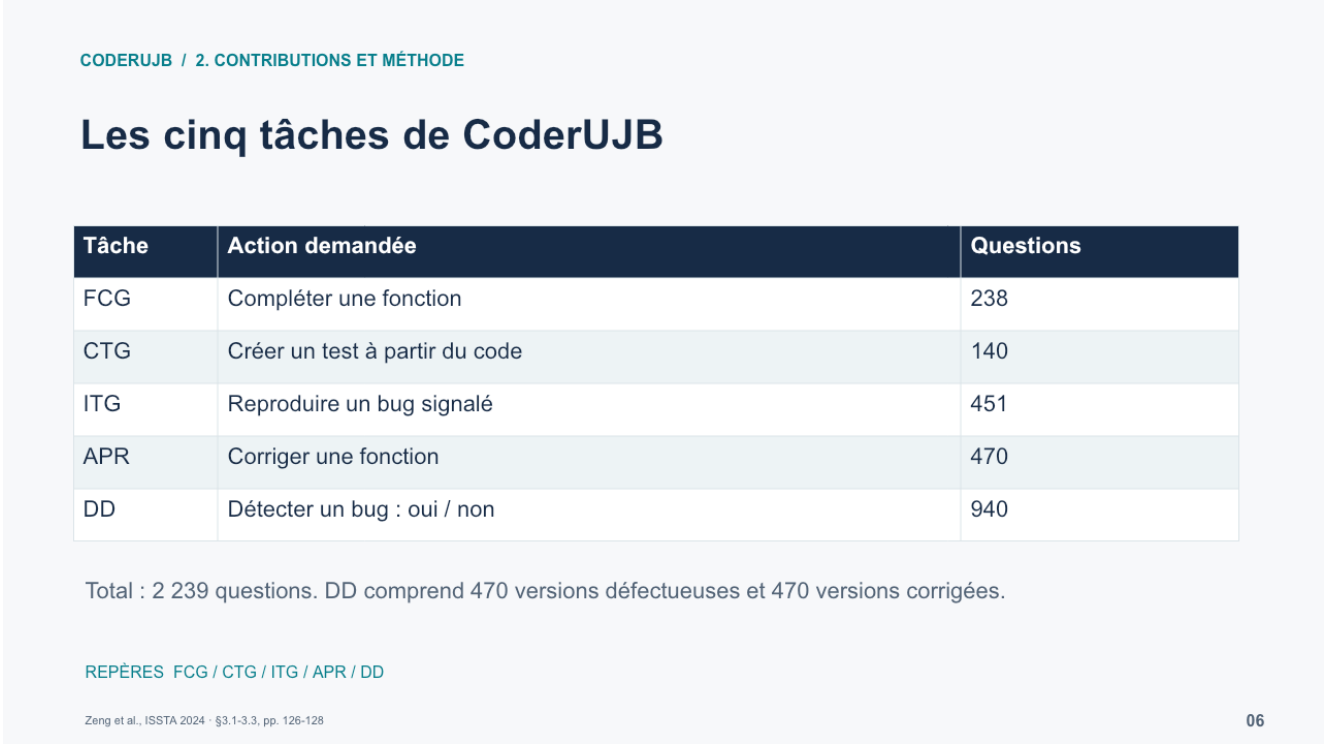
CoderUJB contient deux mille deux cent trente-neuf questions. Elles viennent de dix-sept projets Java disponibles dans Defects4J. Cette source contient des programmes, des bugs et des versions corrigées.

Le premier apport est de réunir cinq tâches dans un même cadre. Le deuxième est de fournir le projet et son environnement d’exécution. Le troisième est une étude comparative de plusieurs modèles.

Repères : 2 239 questions / 17 projets / Defects4J

Source : Résumé, §3.2, pp. 124, 127.

06. Les cinq tâches de CoderUJB



CODERUJB / 2. CONTRIBUTIONS ET MÉTHODE

Les cinq tâches de CoderUJB

Tâche	Action demandée	Questions
FCG	Compléter une fonction	238
CTG	Créer un test à partir du code	140
ITG	Reproduire un bug signalé	451
APR	Corriger une fonction	470
DD	Détecter un bug : oui / non	940

Total : 2 239 questions. DD comprend 470 versions défectueuses et 470 versions corrigées.

REPÈRES FCG / CTG / ITG / APR / DD

Zeng et al., ISSTA 2024 · §3.1-3.3, pp. 126-128 06

TEXTE À DIRE

Voici les cinq tâches. La première consiste à compléter une fonction à partir de sa description. La deuxième demande d’écrire un test à partir du code. La troisième part d’un signalement de bug : le test doit reproduire ce problème.

La quatrième consiste à corriger une fonction défectueuse. Enfin, la cinquième demande de dire si une fonction contient un bug.

Les deux tâches de test ont donc des entrées différentes : le code dans un cas, le signalement du problème dans l’autre.

Repères : FCG / CTG / ITG / APR / DD

Source : §3.1-3.3, pp. 126-128.

07. Du projet à la réponse du modèle



TEXTE À DIRE

Ils extraient le code et les tests, puis appliquent des filtres et une vérification manuelle. Pour la génération de fonctions, ils demandent notamment une couverture de tests supérieure à cinquante pour cent.

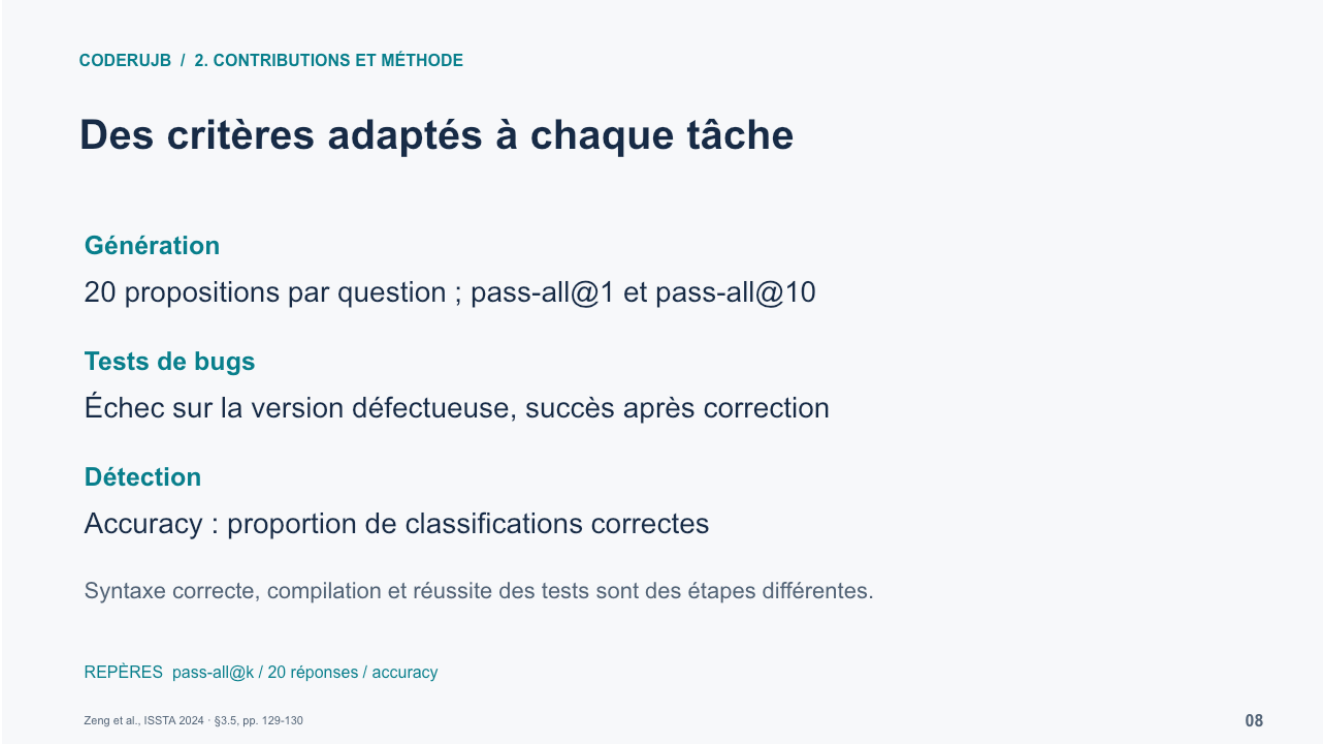
Ensuite, chaque question contient une consigne et les informations utiles à la tâche. Selon le cas, ce sont des imports, des signatures ou un rapport de bug.

Attention : le benchmark fournit le projet complet, mais le prompt ne contient pas forcément tout ce projet. Les solutions générées sont ensuite vérifiées dans leur environnement.

Repères : sélection / contexte / réponse / vérification

Source : §3.3-3.4, pp. 127-129.

08. Des critères adaptés à chaque tâche



TEXTE À DIRE

Pour les tâches de génération, les auteurs produisent vingt réponses par question. Le score pass-all à k estime la probabilité d'avoir au moins une réponse correcte parmi k propositions. Ils présentent notamment k égal à un et à dix.

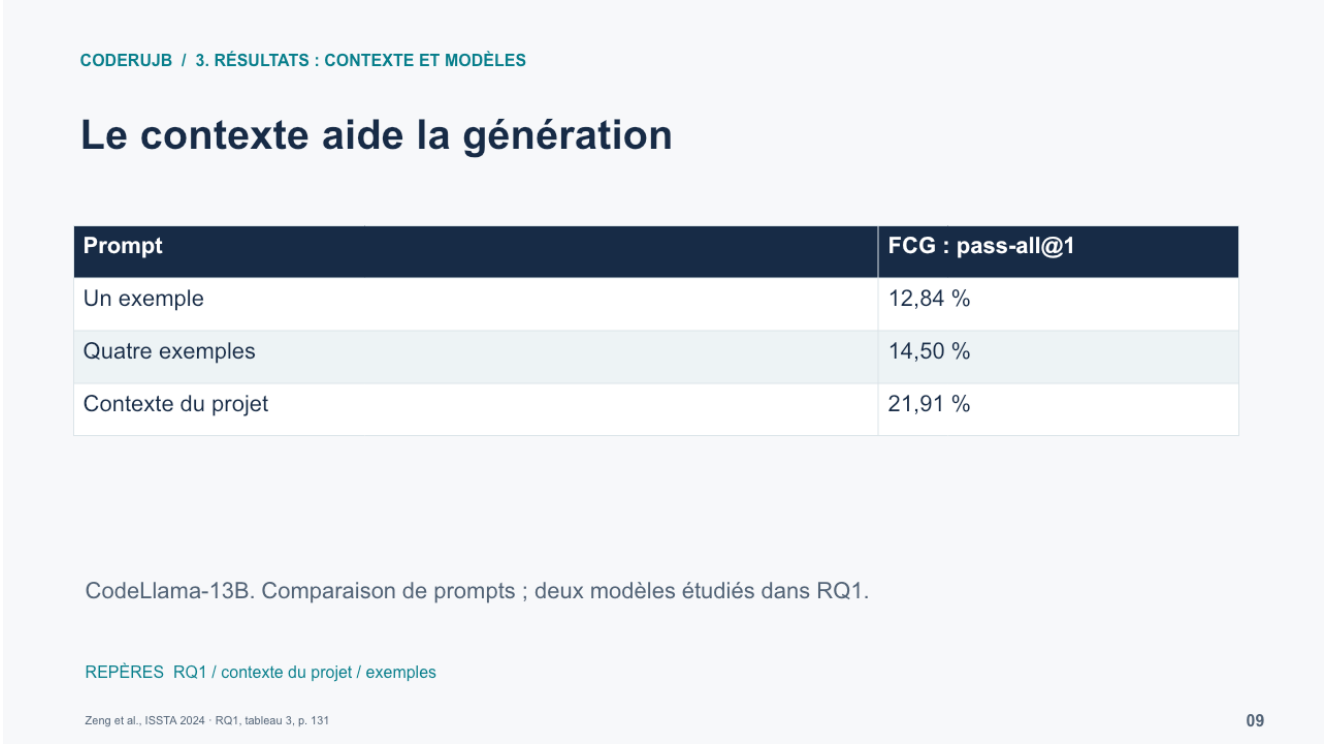
Pour une fonction ou une réparation, la réponse doit passer les tests. Pour reproduire un bug, le test doit échouer sur la version défectueuse et réussir sur la version corrigée. La détection utilise le pourcentage de réponses correctes.

Il faut donc comparer des scores de même nature. La personne suivante va présenter les premiers résultats.

Repères : pass-all@k / 20 réponses / accuracy

Source : §3.5, pp. 129-130.

09. Le contexte aide la génération



TEXTE À DIRE

Je vais répondre à deux questions : le contexte aide-t-il le modèle, et comment les modèles se comparent-ils ?

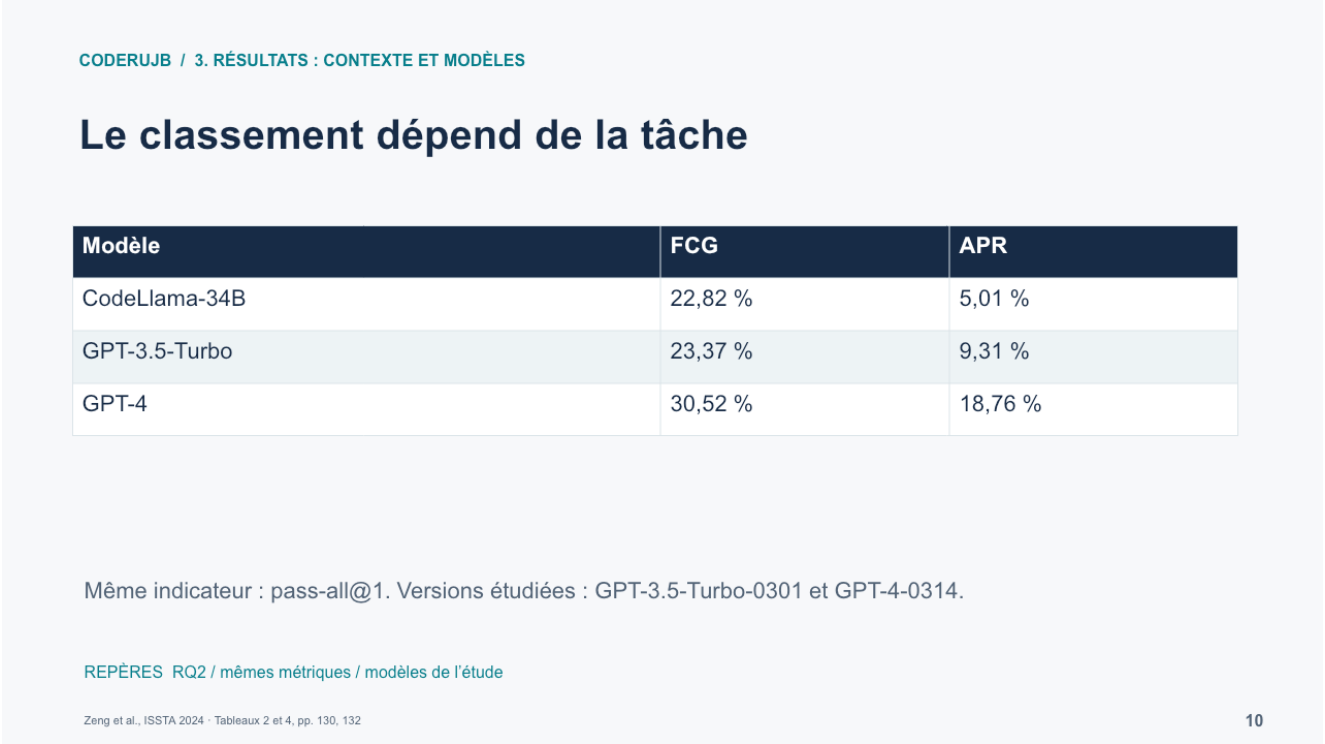
Pour la première question, les auteurs testent deux modèles. Ils comparent un prompt avec du contexte du projet à des prompts contenant un ou quatre exemples.

Pour CodeLlama treize milliards, le score de génération de fonctions atteint environ vingt-deux pour cent avec le contexte. Il est d’environ treize pour cent avec un exemple. Mais ce résultat ne signifie pas qu’il améliore automatiquement toutes les tâches.

Repères : RQ1 / contexte du projet / exemples

Source : RQ1, tableau 3, p. 131.

10. Le classement dépend de la tâche



TEXTE À DIRE

Passons à la comparaison des modèles. Ici, les scores utilisent pass-all à un.

En génération de fonctions, CodeLlama trente-quatre milliards est proche de GPT trois point cinq : environ vingt-trois pour cent dans les deux cas. GPT quatre obtient environ trente et un pour cent.

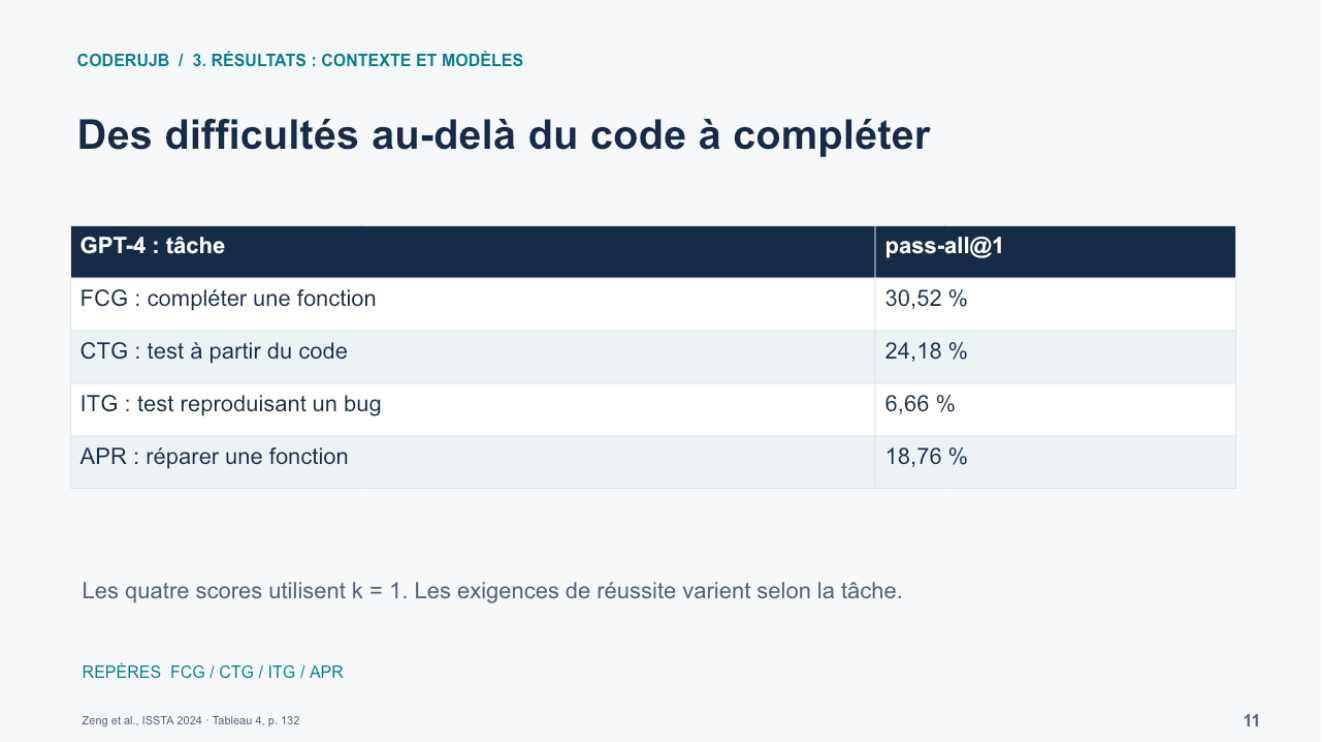
En réparation, l'écart est plus marqué. CodeLlama est à environ cinq pour cent, GPT trois point cinq à neuf, et GPT quatre à dix-neuf.

Ces résultats concernent les versions testées dans l'article. Ils ne constituent pas un classement des modèles actuels.

Repères : RQ2 / mêmes métriques / modèles de l'étude

Source : Tableaux 2 et 4, pp. 130, 132.

11. Des difficultés au-delà du code à compléter



TEXTE À DIRE

Même le meilleur modèle de ce tableau rencontre des difficultés. Pour GPT quatre, le score est d’environ trente et un pour cent pour compléter une fonction. Il descend à vingt-quatre pour cent pour créer un test à partir du code.

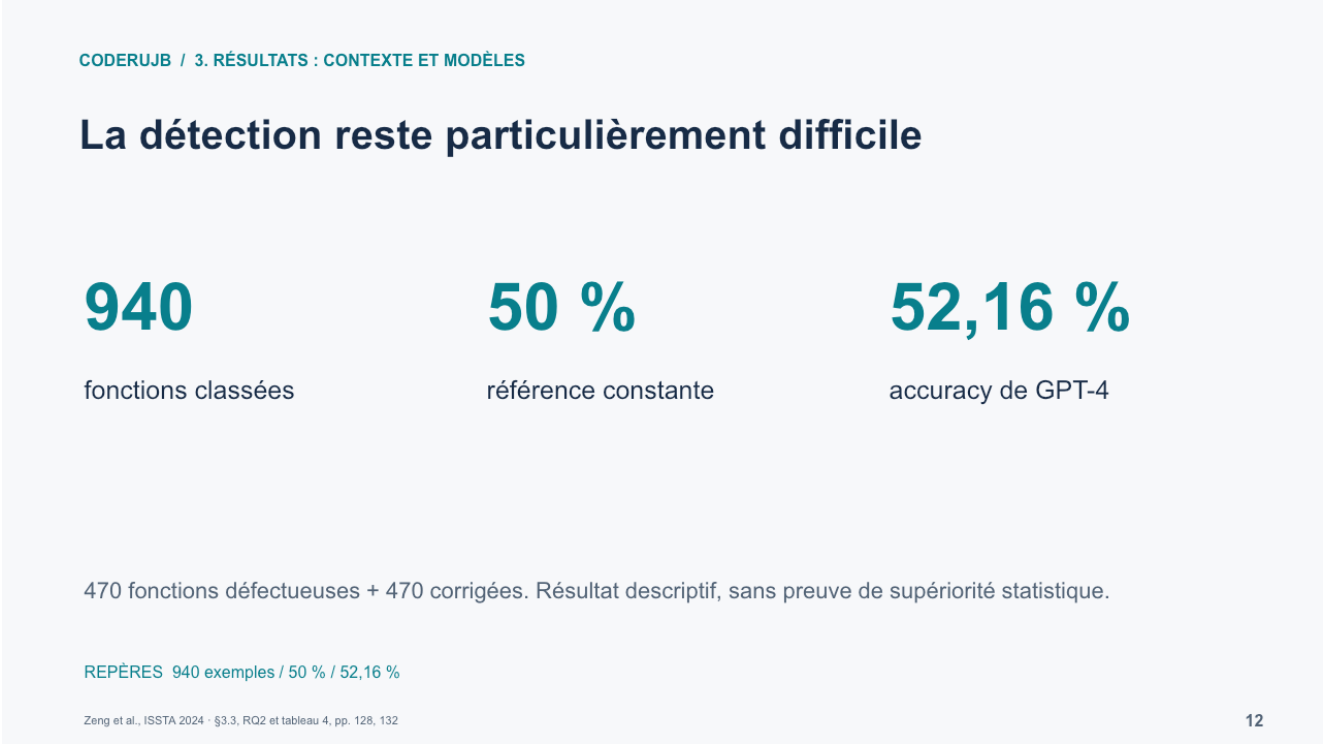
Pour produire un test qui reproduit un bug signalé, il atteint seulement six virgule soixante-six pour cent. La réparation atteint environ dix-neuf pour cent.

Les tâches et les critères diffèrent, donc il faut rester prudent dans la comparaison. Mais ces résultats montrent qu’un bon score en génération de fonctions ne résume pas toutes les capacités.

Repères : FCG / CTG / ITG / APR

Source : Tableau 4, p. 132 ; valeurs choisies plutôt que le texte contradictoire de RQ2.

12. La détection reste particulièrement difficile



TEXTE À DIRE

La détection est encore un autre problème. Le jeu de données contient autant de fonctions défectueuses que de fonctions corrigées. Une réponse toujours identique donnerait donc cinquante pour cent de bonnes réponses.

GPT quatre obtient cinquante-deux virgule seize pour cent. C’est proche de cette référence. Certains modèles produisent aussi des réponses mal formatées.

Le message principal est donc de regarder les résultats tâche par tâche. Je passe maintenant aux effets de l’entraînement, qui peuvent aussi changer selon la tâche.

Repères : 940 exemples / 50 % / 52,16 %

Source : §3.3, RQ2 et tableau 4, pp. 128, 132.

13. Deux façons de poursuivre l'entraînement



TEXTE À DIRE

Je vais maintenant expliquer les effets de deux types d'entraînement.

Le premier est le préentraînement supplémentaire dans un langage précis. Par exemple, on continue à entraîner le modèle avec du code Python ou Java.

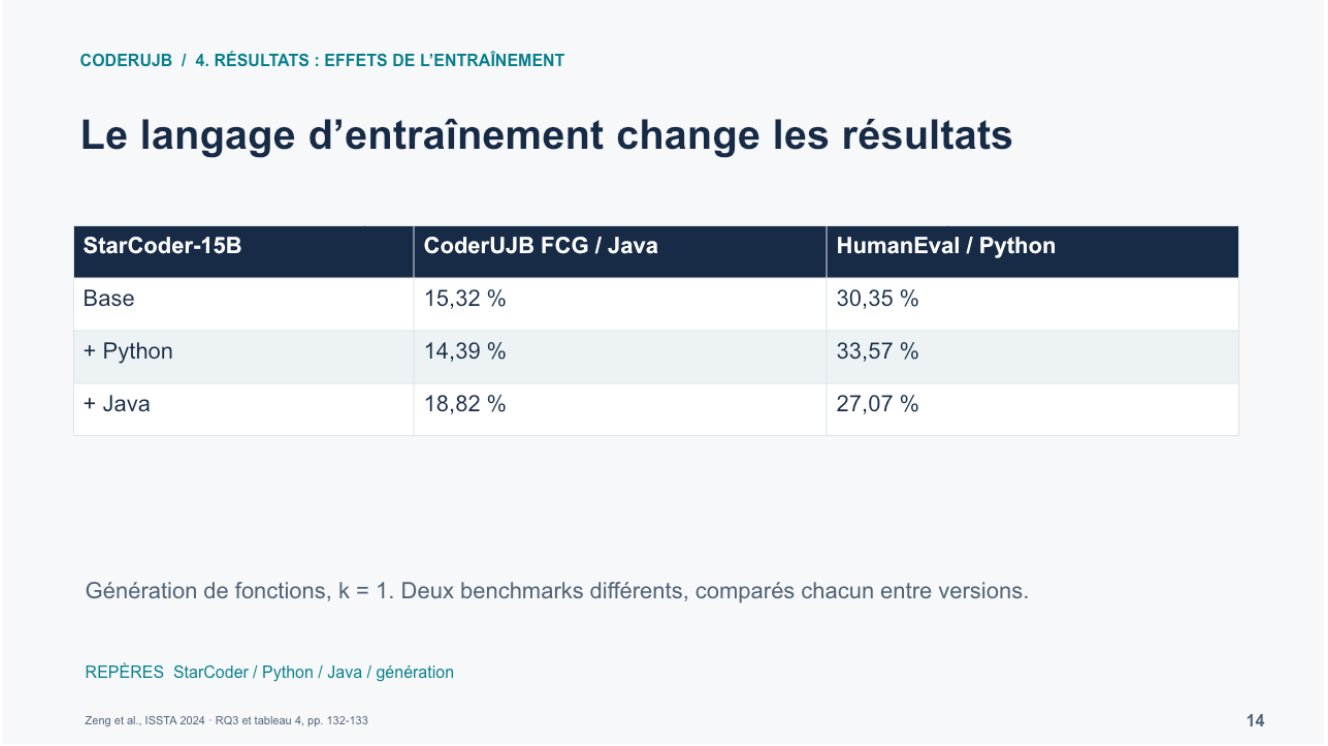
Le second est l'ajustement sur des instructions. On entraîne alors le modèle à répondre à des demandes avec des exemples de tâches et de réponses.

Ils comparent ces versions aux modèles de départ : quelles tâches progressent ?

Repères : RQ3 / langage / RQ4 / instructions

Source : §4.1-4.2, RQ3-RQ4, pp. 130, 133.

14. Le langage d'entraînement change les résultats



TEXTE À DIRE

Voici un exemple avec StarCoder. Sur la génération de fonctions Java de CoderUJB, le modèle de base obtient environ quinze pour cent. La version entraînée davantage sur Python descend à environ quatorze pour cent. La version Java monte à environ dix-neuf pour cent.

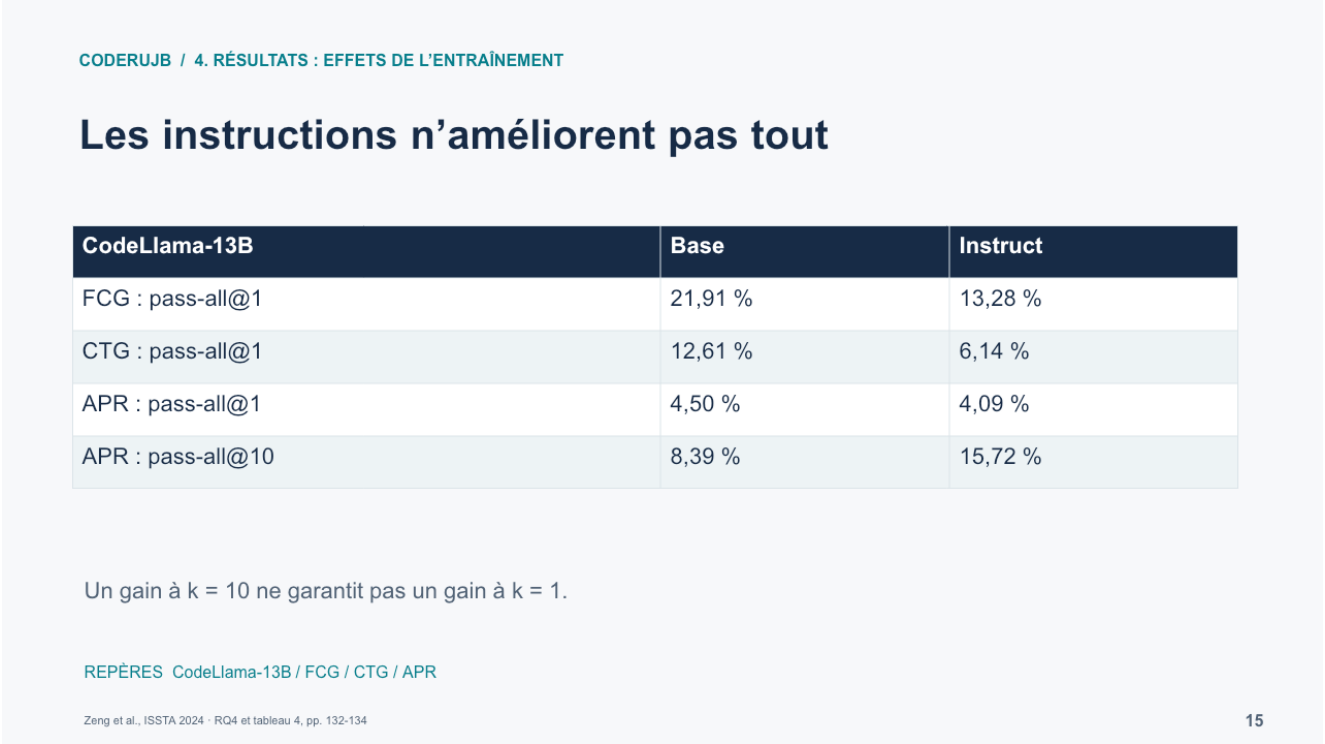
Sur HumanEval en Python, on observe le mouvement inverse : la version Python progresse et la version Java recule.

Cela illustre un compromis entre langages. Mais pour reproduire des bugs ou réparer des fonctions, les auteurs observent des effets moins réguliers. Il ne faut donc pas généraliser cet exemple à toutes les tâches.

Repères : StarCoder / Python / Java / génération

Source : RQ3 et tableau 4, pp. 132-133.

15. Les instructions n'améliorent pas tout



TEXTE À DIRE

L'ajustement sur instructions donne aussi des résultats contrastés. Regardons CodeLlama treize milliards et sa version Instruct.

Pour compléter une fonction, le score à une tentative passe d'environ vingt-deux à treize pour cent. Pour créer un test à partir du code, il passe d'environ treize à six pour cent.

Pour la réparation, il faut préciser le nombre de tentatives. Avec une seule proposition, le score baisse légèrement. Avec dix propositions, il passe d'environ huit à seize pour cent.

Dire simplement que les instructions améliorent la réparation serait donc trop général. La métrique compte aussi.

Repères : CodeLlama-13B / FCG / CTG / APR

Source : RQ4 et tableau 4, pp. 132-134.

16. Une conclusion liée au protocole étudié



TEXTE À DIRE

Les auteurs proposent une explication : l'effet de l'entraînement pourrait dépendre de la proximité entre la tâche évaluée et la tâche de préentraînement. C'est une interprétation des résultats, pas une règle démontrée pour tous les modèles.

CodeLlama Instruct trente-quatre milliards produit des réponses identiques pour différentes questions dans certaines tâches. Les auteurs parlent d'un effondrement du mode de génération et écartent ces résultats de leur analyse correspondante.

Nous retenons donc qu'il faut tester chaque stratégie sur plusieurs tâches. La personne suivante va préciser les limites de cette étude.

Repères : effets variables / hypothèse / prudence

Source : RQ3-RQ4, tableau 5 et discussion, pp. 133-134.

17. Les limites reconnues par les auteurs



TEXTE À DIRE

Je vais maintenant présenter les limites de l’étude. Je distingue celles que les auteurs discutent explicitement et les réserves que nous ajoutons.

Dans leur section sur la validité, les auteurs présentent trois types de risques. Le premier concerne les erreurs possibles dans les scripts. Le deuxième concerne le choix des modèles et la portée des conclusions. Le troisième concerne les indicateurs utilisés.

Les auteurs cherchent à réduire ces risques, sans pouvoir les supprimer.

Repères : validité interne / externe / des mesures

Source : §6, p. 134.

18. Les scripts et les tests restent imparfaits



TEXTE À DIRE

Pour les erreurs d’implémentation, les auteurs expliquent qu’ils ont relu leurs scripts. Ils publient également le code et les résultats pour permettre des vérifications.

Concernant les mesures, ils utilisent plusieurs indicateurs et vérifient la couverture des tests. C’est utile, mais un programme qui passe les tests peut encore contenir un bug non détecté.

De même, couvrir une ligne signifie qu’un test l’a exécutée. Cela ne garantit pas que le test vérifie correctement tous les comportements. Il faut donc comprendre les scores comme une réussite selon les critères du benchmark, pas comme une preuve de correction totale.

Repères : code public / tests / couverture

Source : §3.5 et §6, pp. 129-130, 134 ; distinction couverture/correction : analyse du groupe.

19. Un périmètre précis de projets et de modèles



TEXTE À DIRE

Les auteurs choisissent des modèles qu’ils considèrent comme représentatifs. Mais ce sont des versions précises, évaluées dans les conditions de cette étude.

Nous ajoutons une réserve sur les données. Les questions proviennent de dix-sept projets Java d’une même source. Cela facilite une construction cohérente, mais ne représente pas tous les langages ni tous les logiciels.

De plus, la réparation se concentre sur des défauts dans une seule fonction. On ne peut donc pas supposer que les résultats seraient identiques pour une correction répartie dans plusieurs fichiers.

Repères : Java / 17 projets / versions testées

Source : §3.2-3.3, §4.2, §6, pp. 127-130, 134 ; portée des projets : analyse du groupe.

20. Un protocole de réponse relativement simple



TEXTE À DIRE

Enfin, les auteurs étudient principalement des réponses à des questions ou des complétions de code. Ils ne mesurent pas toute l'activité d'un développeur assisté par une IA.

Par exemple, une méthode qui exécute ses propres tests puis corrige plusieurs fois sa réponse pourrait se comporter autrement.

Il faut aussi distinguer le projet disponible dans le benchmark et le contexte réellement fourni au modèle. Une information absente du prompt peut influencer le résultat.

Ces limites précisent donc où les conclusions s'appliquent. La dernière partie va proposer notre évaluation et des améliorations possibles.

Repères : prompt / réponse / outils / portée

Source : §3.4 et conclusion 2 de RQ2, pp. 128-129, 132 ; portée du protocole : analyse du groupe.

21. Notre évaluation : un apport utile



TEXTE À DIRE

Pour terminer, voici notre évaluation de l'article. Son principal point fort est de rapprocher l'évaluation de plusieurs activités réelles de développement.

Les auteurs réunissent plusieurs tâches, utilisent des projets existants et vérifient les solutions avec des critères adaptés. Ils rendent aussi leurs données et leurs résultats accessibles.

Nous apprécions surtout le fait de regarder les résultats par tâche. Un seul score peut masquer des différences importantes. Cet apport concerne aussi la manière d'évaluer les futurs modèles.

Repères : forces / tâches / exécution / transparence

Source : §3, §5-7, pp. 126-130, 134 ; appréciation du groupe.

22. Des comparaisons à mieux isoler



TEXTE À DIRE

Notre première réserve concerne l'interprétation des différences entre modèles. Les modèles de base utilisent un format de complétion. Les modèles ajustés sur instructions utilisent un format de dialogue.

Les auteurs gardent le même contenu et considèrent ces formats comme équivalents. Mais le format peut aussi influencer la réponse. Nous voudrions donc une expérience supplémentaire qui sépare mieux l'effet de l'entraînement et celui du prompt.

Nous proposerions aussi plusieurs répétitions et des intervalles d'incertitude. Ce sont nos propositions pour vérifier la stabilité des écarts.

Repères : format du prompt / entraînement / protocole

Source : §3.4, §4.2, pp. 128, 130 ; critique et propositions du groupe.

23. Une lecture attentive des chiffres



TEXTE À DIRE

Autre remarque : certains chiffres sont incohérents. Pour GPT quatre, le tableau quatre donne six virgule soixante-six pour cent en génération de tests à partir d'un bug, avec une tentative.

Le texte du résultat deux mentionne quinze virgule soixante-seize pour cent dans une liste présentée comme des scores à une tentative. Or ce chiffre correspond à dix tentatives dans le tableau.

Nous avons donc utilisé les valeurs du tableau et indiqué k sur nos diapositives. Cette incohérence mérite une correction, mais elle ne suffit pas, à elle seule, à invalider l'ensemble de l'étude.

Repères : tableau 4 / k = 1 / k = 10

Source : RQ2, p. 131 et tableau 4, p. 132 ; incohérence vérifiée par le groupe.

24. Conclusion et pistes de prolongement



TEXTE À DIRE

Pour prolonger cette étude, nous proposerions des projets plus variés, d'autres langages et une vérification du risque de présence des exemples dans les données d'entraînement. Nous testerions aussi des méthodes capables d'utiliser des outils et de corriger leurs réponses.

En conclusion, CoderUJB montre l'intérêt d'évaluer les modèles dans un contexte de projet et sur plusieurs tâches. Les effets de l'entraînement varient, et les indicateurs doivent être lus avec précision.

Notre bilan est donc positif, avec des réserves sur la portée et sur certaines comparaisons. Merci pour votre attention. Nous sommes prêts à répondre à vos questions.

Repères : plus de diversité / contrôles / évaluation par tâche

Source : §5-7, p. 134 ; prolongements et bilan du groupe.

Repères pour la répétition

术语与指标

FCG：补全函数。CTG：根据代码生成测试。ITG：根据 bug 报告生成复现测试。APR：自动修复。DD：检测是否存在缺陷。
pass-all@1 与 pass-all@10：至少有一个合格答案的估计概率，分别对应抽取 1 个或 10 个候选。实验生成 20 个候选。DD 使用 accuracy，不与生成任务的指标混为一谈。

第 3、4 位的结果读法

一律先说任务、模型与指标，再读数字。表中的百分比无需全部念，讲稿已挑选重点。
CodeLlama-13B 的 APR 在 k=1 时略降，在 k=10 时上升；不能说所有微调模型、所有指标都变好。

第 5、6 位的边界

第 5 位解释结论的适用范围与风险。第 6 位评价研究方法、比较设计、报告一致性，并提出改进。用“les auteurs”介绍作者观点，用“nous proposons / notre réserve”介绍小组分析。

数字核对

GPT-4 的 ITG：表 4 给出 pass-all@1 = 6,66 %；pass-all@10 = 15,76 %。RQ2 正文把后者放在一组称为 k=1 的结果里。本套材料使用表 4 的数据并保留明确指标。

排练步骤

每个人单独计时一遍，再按 1 到 6 的顺序合练。换页不要重念 PPT 全文。超过 4 分钟时，先删可选例子和重复说明，不要删指标条件或关键结论。每人目标留出 10 秒以上余量。

Référence

Zeng, Z., Wang, Y., Xie, R., Ye, W., & Zhang, S. (2024). CoderUJB: An Executable and Unified Java Benchmark for Practical Programming Scenarios. ISSTA '24, pp. 124-136.
DOI : 10.1145/3650212.3652115